

CS214 Written Assignment HW1

Hugo Wan, GitHub ID: HugoWan0504

Deadline: 4/20/2026

1 Work and Span

Key Rules (from lecture slides)

Basic definitions:

- Work (W): total number of operations
- Span (D): longest dependency chain (critical path)

Parallel rules:

- `parallel_do`
 - $W = W_1 + W_2$
 - $D = \max(D_1, D_2)$
- `parallel_for` (with n iterations)
 - $W = O(n)$
 - $D = O(\log n)$

1.1 (1)

```
void compute(int n) {  
    parallel_for i = 0 to n {  
        parallel_for i = 0 to n {  
            do_something();  
        } } }  
}
```

Work. The inner `parallel_for` runs n iterations, each doing $O(1)$ work, so it costs $O(n)$. Since the outer `parallel_for` also runs n times, the total work is:

$$W(n) = n \cdot O(n) = \Theta(n^2).$$

Span. A `parallel_for` takes $O(\log n)$ span. Because the loops are nested, their spans add up:

$$D(n) = O(\log n) + O(\log n) = \Theta(\log n).$$

Answer: $W(n) = \Theta(n^2)$, $D(n) = \Theta(\log n)$.

1.2 (2)

```
void compute(int n) {
    if (n == 0) return;
    parallel_do {
        compute(n/8);
        compute(n/8);
    }
    do_something();
}
```

Work. Each call makes two recursive calls on size $n/8$, plus constant extra work:

$$W(n) = 2W(n/8) + O(1).$$

Using the Master Theorem, since $\log_8 2 = 1/3$, we get:

$$W(n) = \Theta(n^{1/3}).$$

Span. The two recursive calls run in parallel, so only one contributes to the span:

$$D(n) = D(n/8) + O(1).$$

We keep dividing n by 8 each time, so:

$$D(n) = \Theta(\log n).$$

Answer: $W(n) = \Theta(n^{1/3})$, $D(n) = \Theta(\log n)$.

1.3 (3)

```
void compute(int n) {
    if (n == 0) return;
    parallel_do {
        compute(n/2);
        compute(n/2);
    }
    parallel_for i = 0 to sqrt(n) {
        do_something();
    }
}
```

Work. The recurrence is:

$$W(n) = 2W(n/2) + O(\sqrt{n}).$$

Since $\sqrt{n}$ grows slower than n , this is dominated by the $2W(n/2)$ part. So,

$$W(n) = \Theta(n).$$

Span. The recursive calls run in parallel, but the `parallel_for` over $\sqrt{n}$ adds $O(\log n)$ span:

$$D(n) = D(n/2) + O(\log n).$$

Solving this gives:

$$D(n) = \Theta(\log^2 n).$$

Answer: $W(n) = \Theta(n)$, $D(n) = \Theta(\log^2 n)$.

1.4 (4) Bonus

```
void compute(int n) {  
    if (n < 4) return;  
    parallel_for i = 0 to sqrt(n) { compute(sqrt(n)); }  
    parallel_for i = 0 to n {  
        do_something();  
    }  
}
```

Work. The first `parallel_for` has $\sqrt{n}$ iterations, and each iteration makes a recursive call on input size $\sqrt{n}$. So this contributes

$$\sqrt{n} W(\sqrt{n}).$$

The second `parallel_for` does n constant-cost operations, so it contributes $O(n)$ work. Therefore,

$$W(n) = \sqrt{n} W(\sqrt{n}) + O(n).$$

To solve it, notice that at the top level the non-recursive work is $O(n)$. At the next level, there are $\sqrt{n}$ subproblems of size $\sqrt{n}$, so the total non-recursive work is

$$\sqrt{n} \cdot O(\sqrt{n}) = O(n).$$

At the next level again, the total is still $O(n)$. So each level costs $O(n)$ work.

The recursion stops when the problem size becomes smaller than 4. Since the size changes as

$$n \rightarrow \sqrt{n} \rightarrow n^{1/4} \rightarrow n^{1/8} \rightarrow \dots,$$

there are $\Theta(\log \log n)$ levels. Therefore,

$$W(n) = \Theta(n \log \log n).$$

Span. For the first `parallel_for`, all recursive calls run in parallel, so the span is the span of one recursive call plus the cost of the loop itself:

$$D(\sqrt{n}) + O(\log n).$$

The second `parallel_for` adds another $O(\log n)$ span. So

$$D(n) = D(\sqrt{n}) + O(\log n).$$

Expanding the recurrence gives

$$\begin{aligned} D(n) &= O(\log n) + O(\log \sqrt{n}) + O(\log n^{1/4}) + \dots \\ &= O\left(\log n + \frac{1}{2} \log n + \frac{1}{4} \log n + \dots\right) = O(\log n). \end{aligned}$$

So,

$$D(n) = \Theta(\log n).$$

Answer: $W(n) = \Theta(n \log \log n)$, $D(n) = \Theta(\log n)$.

1.5 (5) Bonus

```
void compute(int n) {
    if (n < 4) return;
    parallel_for i = 0 to sqrt(n) { do_something(); }
    compute(n - sqrt(n));
}
```

Work. The `parallel_for` over $\sqrt{n}$ iterations does constant work in each iteration, so it contributes $O(\sqrt{n})$ work. Then there is one recursive call on size $n - \sqrt{n}$. Thus,

$$W(n) = W(n - \sqrt{n}) + O(\sqrt{n}).$$

To understand this recurrence better, let the current problem size after each recursive call be $n_0, n_1, n_2, \dots$, where

$$n_{i+1} = n_i - \sqrt{n_i}.$$

Now look at $\sqrt{n_i}$. If we let $x_i = \sqrt{n_i}$, then

$$x_{i+1} = \sqrt{x_i^2 - x_i}.$$

This means x_i decreases by a constant amount each time, roughly between 1/2 and 1, so the number of recursive calls is $\Theta(\sqrt{n})$.

Also, the work done at each level is $\Theta(\sqrt{n_i}) = \Theta(x_i)$. Since x_i decreases roughly linearly from $\sqrt{n}$ down to a constant, the total work is like summing an arithmetic series:

$$\Theta(\sqrt{n}) + \Theta(\sqrt{n} - 1) + \Theta(\sqrt{n} - 2) + \dots = \Theta(n).$$

So,

$$W(n) = \Theta(n).$$

Span. The `parallel_for` contributes $O(\log n)$ span, and then the recursive call happens afterward, so

$$D(n) = D(n - \sqrt{n}) + O(\log n).$$

From the work analysis above, there are $\Theta(\sqrt{n})$ recursive levels. Each level contributes at most $O(\log n)$ span, so

$$D(n) = O(\sqrt{n} \log n).$$

Also, for many of those levels the input size is still polynomially related to n , so each level contributes at least a constant fraction of $\log n$ for $\Theta(\sqrt{n})$ levels. Hence the bound is tight:

$$D(n) = \Theta(\sqrt{n} \log n).$$

Answer: $W(n) = \Theta(n)$, $D(n) = \Theta(\sqrt{n} \log n)$.

2 (Another) Parallel Scan Algorithm

2.1 (1)

Divide-and-conquer scan.

Work.

$$W(n) = 2W(n/2) + O(1) = \Theta(n).$$

Span.

$$D(n) = D(n/2) + O(1) = \Theta(\log n).$$

Decrease-and-conquer scan.

Work.

$$W(n) = W(n/2) + O(n) = \Theta(n).$$

Span.

$$D(n) = D(n/2) + O(\log n).$$

Expanding the recurrence:

$$D(n) = \log n + \log(n/2) + \log(n/4) + \dots = \Theta(\log^2 n).$$

Conclusion. Both have $\Theta(n)$ work, but divide-and-conquer has better span: $\Theta(\log n)$ vs. $\Theta(\log^2 n)$.

—

2.2 (2)

To compute the values in Step 3, we take the prefix sum of the array $S[1..k]$. That is, for each chunk j , we compute

$$P[j] = \sum_{t=1}^j S[t].$$

Then the value at the end of chunk j is just $P[j]$.

—

2.3 (3)

For Step 4, each chunk is processed sequentially, but all chunks run in parallel.

For chunk j , we use the prefix sum of all previous chunks as an offset:

$$\text{offset} = P[j - 1].$$

Then we compute the prefix sums inside the chunk starting from this offset.

—

2.4 (4)

Work.

Step 2 and Step 4 each process all n elements (split across k chunks), so each costs $O(n)$ work. Step 3 processes k elements sequentially, costing $O(k)$.

So the total work is

$$W(n) = O(n) + O(k) + O(n) = O(n + k).$$

Since $k \leq n$, this simplifies to

$$W(n) = \Theta(n).$$

—

2.5 (5)

Span.

Step 2: k chunks run in parallel, each taking $\Theta(n/k)$ time Step 3: sequential over k elements $\rightarrow \Theta(k)$ Step 4: same as Step 2 $\rightarrow \Theta(n/k)$

So,

$$D(n) = \Theta(n/k + k + n/k) = \Theta(n/k + k).$$

—

2.6 (6)

To minimize the span

$$D(n) = \Theta(n/k + k),$$

we balance the two terms:

$$n/k = k \Rightarrow k = \Theta(\sqrt{n}).$$

Plugging back in:

$$D(n) = \Theta(\sqrt{n}).$$

—

2.7 (7) Bonus

If $k = \Theta(n)$, then each chunk has constant size.

From (5),

$$D(n) = \Theta(n/k + k) = \Theta(1 + n) = \Theta(n),$$

which is very large.

The issue is that Step 3 is still sequential over $k = \Theta(n)$ elements.

To fix this, we can replace Step 3 with a **parallel scan** on S . After this change, the algorithm essentially becomes the decrease-and-conquer scan we learned in class.

References

- Professor Yihan Sun, *Implementing Parallel Algorithms*, CS214 Lecture Slides.
- Professor Yihan Sun, *Our First Parallel Algorithm*, CS214 Lecture Slides.
- Professor Yihan Sun, *Parallel Sequence Algorithms*, CS214 Lecture Slides.
- OpenAI. *ChatGPT (GPT-5.3)*. Used for suggesting alternative parallel optimization strategies and provided pseudo-code idea.
<https://chatgpt.com/share/69e41716-e4e4-8327-acbe-05de74025eb4>